“The Game Beneath the Game: The Evolution of Cheats and Anti-Cheat Systems in Counter-Strike 2”

Author: Pablo Valiente

Supervisor: Dr. Antonio Nappa

Universidad Carlos III de Madrid

Academic Year: 2024–2025

Outline

Outline

Motivation

My interest in this topic began around six years ago, driven by a passion for competitive video games.

- **Curiosity:** Encountering cheaters in ranked matches often led me to wonder: *"How did they do it?"*
- **Exploration:** This question gradually introduced me to reverse engineering, memory analysis, and anti-cheat evasion.
- **Objective:** To merge this journey with an academic study by analyzing and implementing real cheats to test detection systems.

Motivation

My interest in this topic began around six years ago, driven by a passion for competitive video games.

- **Curiosity:** Encountering cheaters in ranked matches often led me to wonder: *"How did they do it?"*
- **Exploration:** This question gradually introduced me to reverse engineering, memory analysis, and anti-cheat evasion.
- **Objective:** To merge this journey with an academic study by analyzing and implementing real cheats to test detection systems.

Motivation

My interest in this topic began around six years ago, driven by a passion for competitive video games.

- **Curiosity:** Encountering cheaters in ranked matches often led me to wonder: *"How did they do it?"*
- **Exploration:** This question gradually introduced me to reverse engineering, memory analysis, and anti-cheat evasion.
- **Objective:** To merge this journey with an academic study by analyzing and implementing real cheats to test detection systems.

Outline

Understanding Cheating in Online Games

Cheating undermines fairness and security in competitive online games.

- **Definition:** Any software or hardware manipulation that grants unfair advantage.
- **Anti-Cheat:** Monitors and enforces integrity at runtime, acting like an **EDR** by detecting anomalous behavior, unauthorized memory access, or code injection in protected processes.
- **Spectrum:** From basic aimbots to advanced GUI Drawings.

Understanding Cheating in Online Games

Cheating undermines fairness and security in competitive online games.

- **Definition:** Any software or hardware manipulation that grants unfair advantage.
- **Anti-Cheat:** Monitors and enforces integrity at runtime, acting like an **EDR** by detecting anomalous behavior, unauthorized memory access, or code injection in protected processes.
- **Spectrum:** From basic aimbots to advanced GUI Drawings.

Understanding Cheating in Online Games

Cheating undermines fairness and security in competitive online games.

- **Definition:** Any software or hardware manipulation that grants unfair advantage.
- **Anti-Cheat:** Monitors and enforces integrity at runtime, acting like an **EDR** by detecting anomalous behavior, unauthorized memory access, or code injection in protected processes.
- **Spectrum:** From basic aimbots to advanced GUI Drawings.

Historical Origins: From Developer Shortcuts to Cheating Tools

Cheating emerged from tools meant for testing and evolved into manipulation.

- **Debug Codes:** Became public cheat codes (e.g., Konami Code).
- **GameShark:** Modified RAM directly on console hardware.
- **Cheat Engine & ReClass.NET:** Allowed advanced memory mapping, pointer tracing, and class reconstruction.

Historical Origins: From Developer Shortcuts to Cheating Tools

Cheating emerged from tools meant for testing and evolved into manipulation.

- **Debug Codes:** Became public cheat codes (e.g., Konami Code).
- **GameShark:** Modified RAM directly on console hardware.
- **Cheat Engine & ReClass.NET:** Allowed advanced memory mapping, pointer tracing, and class reconstruction.

Historical Origins: From Developer Shortcuts to Cheating Tools

Cheating emerged from tools meant for testing and evolved into manipulation.

- **Debug Codes:** Became public cheat codes (e.g., Konami Code).
- **GameShark:** Modified RAM directly on console hardware.
- **Cheat Engine & ReClass.NET:** Allowed advanced memory mapping, pointer tracing, and class reconstruction.

From Hacks to Services: The Turning Point

The early 2010s marked a major shift in how cheats were created, distributed, and perceived.

- **Community Forums:** Platforms like UnKnoWnCheaTs fostered knowledge sharing and tutorials.
- **Subscription Models:** Private cheats became commercialized with regular updates and support.

From Hacks to Services: The Turning Point

The early 2010s marked a major shift in how cheats were created, distributed, and perceived.

- **Community Forums:** Platforms like UnKnoWnCheaTs fostered knowledge sharing and tutorials.
- **Subscription Models:** Private cheats became commercialized with regular updates and support.

Cheating Today: A Professionalized Ecosystem

Cheating has evolved from hobbyist tools to a mature ecosystem that mirrors legitimate software development.

- **Commercialization:** Cheat providers operate like startups, offering licensing, tiered features, and customer support.
- **Infrastructure:** Loaders, obfuscators, HWID spoofers, and encrypted update channels ensure persistence.

Cheating Today: A Professionalized Ecosystem

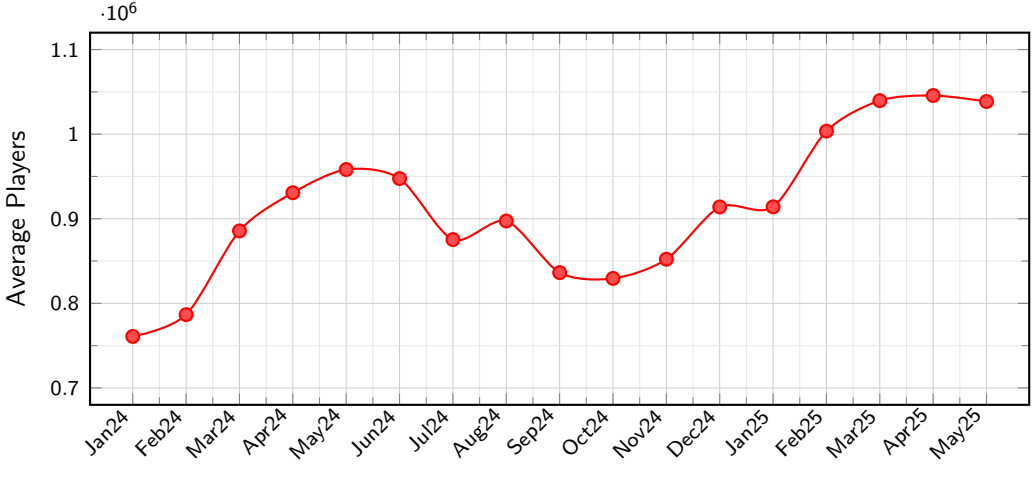
Cheating has evolved from hobbyist tools to a mature ecosystem that mirrors legitimate software development.

- **Commercialization:** Cheat providers operate like startups, offering licensing, tiered features, and customer support.
- **Infrastructure:** Loaders, obfuscators, HWID spoofers, and encrypted update channels ensure persistence.

Outline

Why Counter Strike 2?

To contextualize the scale of CS2's ecosystem, the following graph illustrates the evolution of its player base based on SteamCharts data.



Steam Top 10: July 7, 2025

Top Games By Current Players

	Name	Current Players	Last 30 Days	Peak Players	Hours Played
1.	Counter-Strike 2	1,222,782		1,725,815	729,749,131
2.	PUBG: BATTLEGROUNDS	581,616		778,327	226,931,487
3.	Dota 2	508,454		648,095	300,471,647
4.	Bongo Cat	169,028		192,501	110,247,318
5.	Delta Force	164,154		177,775	60,062,183
6.	Apex Legends	150,271		250,796	73,537,334
7.	Stardew Valley	133,590		156,589	47,739,319
8.	Rust	129,532		187,345	73,879,862
9.	Wallpaper Engine	120,513		138,702	61,757,771
10.	Grand Theft Auto V Legacy	90,856		112,862	53,501,676

More

Anti-Cheat Systems Timeline

Anti-cheat systems have evolved from basic client-side tools to complex, kernel-level detectors. Each marked a significant shift in visibility, access, or enforcement.

System	Year	Mode	Milestone or Innovation
PunkBuster	2000	User	First real-time memory scan and screenshot system
VAC	2002	User	Delayed ban model to obfuscate detection methods
BattleEye	2004	Kernel	Early adoption of kernel-mode drivers
FairFight	2012	Server	Behavioral/statistical detection at server level
EAC	2018	Kernel	Real-time scanning with driver-level protection
Vanguard	2020	Kernel	Persistent always-on kernel driver (controversial)
VAC Live	2023	Hybrid	Real-time detection with low-latency enforcement

Survey: Cheating as a Critical Threat

A 2018 global survey by Irdeto revealed the widespread negative impact of cheating in Counter-Strike:

Survey Statement	**Percentage**
Experienced negative impact due to cheating	60%
Would stop playing a game because of cheaters	77%
Support permanent bans for cheaters	69%
Believe developers are not doing enough	55%

- Highlights long-term damage to player trust and retention.
- Economic implications due to reduced engagement and spending.

Outline

VAC Detection and Countermeasures

VAC employs both traditional and modern mechanisms to detect unauthorized modifications within the game environment.

- **Detection Vectors:** Signature scanning, analysis of suspicious thread start addresses, and detection of VMT (Virtual Method Table) hooks are among its key strategies.
- **Steamservice.dll:** Inspection of `steamservice.dll` reveals that VAC dynamically streams detection modules from Valve's servers into memory at runtime.
- **Dumper Feasibility:** With moderate reverse engineering effort, it is possible to create a dumper capable of capturing and writing these modules to disk for offline analysis.

VAC Detection and Countermeasures

VAC employs both traditional and modern mechanisms to detect unauthorized modifications within the game environment.

- **Detection Vectors:** Signature scanning, analysis of suspicious thread start addresses, and detection of VMT (Virtual Method Table) hooks are among its key strategies.
- **Steamservice.dll:** Inspection of `steamservice.dll` reveals that VAC dynamically streams detection modules from Valve's servers into memory at runtime.
- **Dumper Feasibility:** With moderate reverse engineering effort, it is possible to create a dumper capable of capturing and writing these modules to disk for offline analysis.

VAC Detection and Countermeasures

VAC employs both traditional and modern mechanisms to detect unauthorized modifications within the game environment.

- **Detection Vectors:** Signature scanning, analysis of suspicious thread start addresses, and detection of VMT (Virtual Method Table) hooks are among its key strategies.
- **Steamservice.dll:** Inspection of `steamservice.dll` reveals that VAC dynamically streams detection modules from Valve's servers into memory at runtime.
- **Dumper Feasibility:** With moderate reverse engineering effort, it is possible to create a dumper capable of capturing and writing these modules to disk for offline analysis.

Community on VAC

VAC Inspecting Suspicious Threads

Valve Updates Anti Cheat

Outline

Entity List and Memory Traversal

During the reverse engineering process of CS2, we identified how entities are registered and updated in memory.

- **Memory Structures:** Reverse engineering revealed a central list containing pointers to entity classes and their data.
- **Entity Management:** C_CSPlayerPawm & C_CBasePlayerController.
- **Cheat Integration:** Our entity list must stay in sync with the game to avoid crashes from invalid pointers.

Entity List and Memory Traversal

During the reverse engineering process of CS2, we identified how entities are registered and updated in memory.

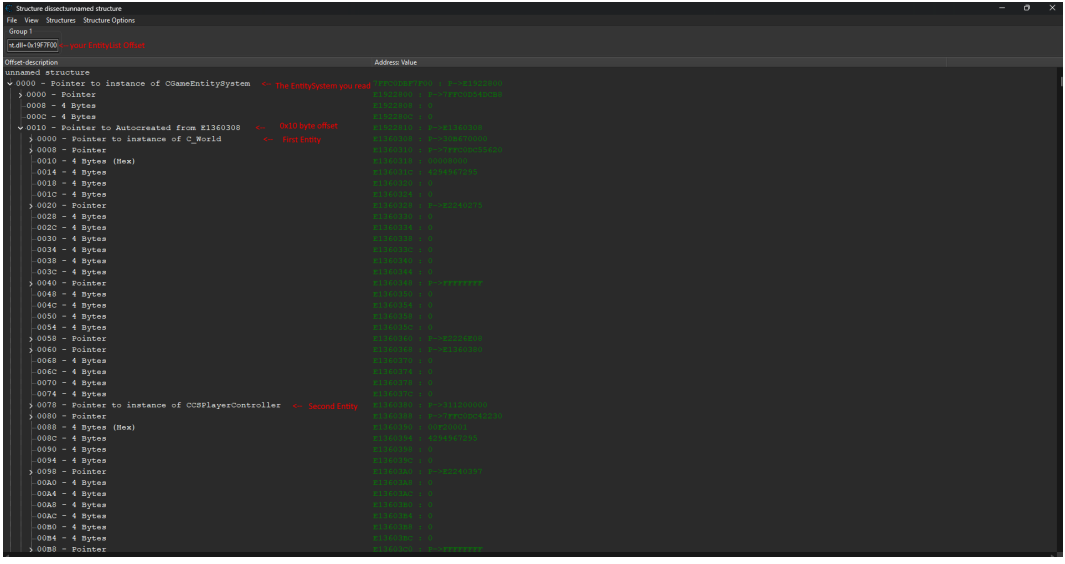
- **Memory Structures:** Reverse engineering revealed a central list containing pointers to entity classes and their data.
- **Entity Management:** C_CSPlayerPawm & C_CBasePlayerController.
- **Cheat Integration:** Our entity list must stay in sync with the game to avoid crashes from invalid pointers.

Entity List and Memory Traversal

During the reverse engineering process of CS2, we identified how entities are registered and updated in memory.

- **Memory Structures:** Reverse engineering revealed a central list containing pointers to entity classes and their data.
- **Entity Management:** C_CSPlayerPawm & C_CBasePlayerController.
- **Cheat Integration:** Our entity list must stay in sync with the game to avoid crashes from invalid pointers.

Entity List Reversed in Reclass every 0x78 Bytes



Feature: Chams & Internal Material Injection

To implement Chams, we reverse engineered one of CS2's internal material files to understand its structure.

- **Material Discovery:** We decrypted and analyzed a material from Valve's asset system (suspected format: `.vk3`) dumped from memory.
- **Function Hooks:** We hooked two internal engine functions: one responsible for material creation, another for rendering passes.
- **Selective Injection:** By filtering the render queue, we applied our custom material only to specific models (e.g. **C_CSPlayerPawn, C_CSWeapon**).

Feature: Chams & Internal Material Injection

To implement Chams, we reverse engineered one of CS2's internal material files to understand its structure.

- **Material Discovery:** We decrypted and analyzed a material from Valve's asset system (suspected format: `.vk3`) dumped from memory.
- **Function Hooks:** We hooked two internal engine functions: one responsible for material creation, another for rendering passes.
- **Selective Injection:** By filtering the render queue, we applied our custom material only to specific models (e.g. **C_CSPlayerPawn, C_CSWeapon**).

Feature: Chams & Internal Material Injection

To implement Chams, we reverse engineered one of CS2's internal material files to understand its structure.

- **Material Discovery:** We decrypted and analyzed a material from Valve's asset system (suspected format: `.vk3`) dumped from memory.
- **Function Hooks:** We hooked two internal engine functions: one responsible for material creation, another for rendering passes.
- **Selective Injection:** By filtering the render queue, we applied our custom material only to specific models (e.g. **C_CSPlayerPawn, C_CSWeapon**).

Material VK3 structure

```
static constexpr char szVMatBufferMetalic[] =
    R"(<!-- kv3 encoding:text:version{e21c7f3c-8a33-41c5-9977-a76d3a32aa0d}
format:generic:version{7412167c-06e9-4698-aff2-e63eb59037e7} -->
{
    Shader = "csgo_complex.vfx"
                F_IGNOREZ = 0
                F_DISABLE_Z_WRITE = 0
                F_DISABLE_Z_BUFFERING = 0
F_RENDER_BACKFACES = 0
    F_TRANSLUCENT = 1
    F_PAINT_VERTEX_COLORS = 1
    g_vColorTint = [1.000000, 1.000000, 1.000000, 1.000000]
    TextureNormal = resource:"materials/default/default_normal.tga"
    g_tAmbientOcclusion = resource:"materials/default/default_ao_tga_559f1ac6.vtex"
    g_tColor = resource:"materials/default/default_color_tga_72dcfbfd.vtex"
    g_tNormal = resource:"materials/default/default_normal_tga_7be61377.vtex"
})";
```

Visual Examples: Chams Materials

Visual Examples: Chams Materials

Visual Examples: Chams Materials

Visual Examples: Chams Materials

Bone ESP: Skeleton Rendering via Bone Structures

During our reverse engineering efforts, we located the in-memory structure used by CS2 to store bone positions for each player model.

- **Bone Matrix Discovery:** The game maintains a matrix of bone positions indexed per entity, storing 3D coordinates for each joint.
- **Bone Indices:** Once the matrix was located, we extracted the relevant joint coordinates and projected them into screen space.
- **Skeleton Drawing:** We connected selected bones (e.g., head, spine, limbs) with lines to visually represent the player's skeleton.

Bone ESP: Skeleton Rendering via Bone Structures

During our reverse engineering efforts, we located the in-memory structure used by CS2 to store bone positions for each player model.

- **Bone Matrix Discovery:** The game maintains a matrix of bone positions indexed per entity, storing 3D coordinates for each joint.
- **Bone Indices:** Once the matrix was located, we extracted the relevant joint coordinates and projected them into screen space.
- **Skeleton Drawing:** We connected selected bones (e.g., head, spine, limbs) with lines to visually represent the player's skeleton.

Bone ESP: Skeleton Rendering via Bone Structures

During our reverse engineering efforts, we located the in-memory structure used by CS2 to store bone positions for each player model.

- **Bone Matrix Discovery:** The game maintains a matrix of bone positions indexed per entity, storing 3D coordinates for each joint.
- **Bone Indices:** Once the matrix was located, we extracted the relevant joint coordinates and projected them into screen space.
- **Skeleton Drawing:** We connected selected bones (e.g., head, spine, limbs) with lines to visually represent the player's skeleton.

Visual Examples: Bone Matrix

Visual Examples: Skeleton

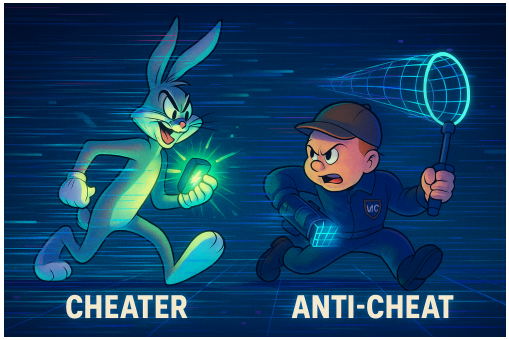
Visual Examples: Skeleton & Chams in a Real Match

Visual Examples: Skeleton in a Real Match

Outline

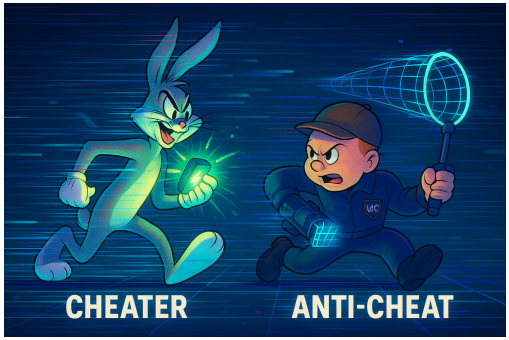
Final Reflections

- The cat-and-mouse dynamic between cheaters and anti-cheat systems reflects a fundamental aspect of human nature: **the relentless drive to win, by any means necessary**.
- A single user, acting anonymously from home, can challenge and undermine the work of entire development teams and multi-million-dollar infrastructures.



Final Reflections

- The cat-and-mouse dynamic between cheaters and anti-cheat systems reflects a fundamental aspect of human nature: **the relentless drive to win, by any means necessary**.
- A single user, acting anonymously from home, can challenge and undermine the work of entire development teams and multi-million-dollar infrastructures.



Future Work and Research Directions

- Integrate automated schema extraction and dynamic offset resolution for update resilience.
- Design and test new stealth hooking methods, including hardware breakpoints and kernel-level indirection.
- Explore and implement hooks from the **Anti Cheat dumped modules**.

Future Work and Research Directions

- Integrate automated schema extraction and dynamic offset resolution for update resilience.
- Design and test new stealth hooking methods, including hardware breakpoints and kernel-level indirection.
- Explore and implement hooks from the **Anti Cheat dumped modules**.

Future Work and Research Directions

- Integrate automated schema extraction and dynamic offset resolution for update resilience.
- Design and test new stealth hooking methods, including hardware breakpoints and kernel-level indirection.
- Explore and implement hooks from the **Anti Cheat dumped modules**.

Thank you for your attention!

Any questions?